个人学科教材 · 修订版

计算机科学基础

信息表示、算法正确性与计算模型

宋体正文 · 数学公式排版

概念、推导与讲解型例题

2026-10-17

目录

1. 信息、符号与编码 3

2. 位置记数法与进制转换 4

3. 整数编码、溢出与解释类型 5

4. 字符、文本与近似数 5

5. 存储程序计算机的组成 6

6. 问题规格、算法与程序 6

7. 循环不变量与终止性 7

8. 时间与空间复杂度 7

9. 状态、字母表与有限自动机 8

10. 有限自动机的构造与正确性 8

11. 有限记忆的边界 10

12. 图灵机的计算机制 10

13. 通用计算与停机问题 11

14. 抽象层次之间的联系 11

1. 信息、符号与编码

屏幕上的字母 A、变量中的整数六十五和一条处理器指令，都可以由二进制位表示。但相同的位串并不自带相同含义：`01000001` 按无符号整数解释为六十五，在 ASCII 字符编码中对应 A；若作为某种指令的一部分，还需要具体指令集和上下文。区分“物理载体、符号表示、解释规则”，是理解计算机的第一步。

比特取零或一，n 个比特共有 2^n 种不同组合。这个结论可由乘法计数原理得到：第一位两种选择，每一种再配第二位的两种选择，依次相乘。组合数说明可区分多少状态，并不规定这些状态分别代表什么。要编码至少一百种对象，六位只有六十四种不够，七位有一百二十八种才够；多出的编码可以保留或表示其他信息。

字节在常见现代设备上为八比特。C 语言中一个字节由 `char` 的存储单位定义，`sizeof(char)` 恒为一，具体位数由 `CHAR_BIT` 给出。因此阅读语言规范时要区分“八位字节的常见机器”和“语言对所有实现的要求”。同理，写成四个字节的整数不自动说明有符号性、字节序或编码方案。

容量分析。 一幅一百乘一百像素的图像，若每像素使用二十四位且不计头部、压缩和对齐，像素数据为 $100 \times 100 \times 24 = 240000$ 比特，即三万字节。图像尺寸、每像素位数和文件大小分别对应不同层次。压缩文件可能小得多，也可能增加其他数据，不能仅从像素数断言实际文件恰有三万字节。

1.1 信号、数据、信息与知识

计算机接收和处理的首先是有物理实现的信号及其编码。温度传感器把环境温度转成电信号，转换电路产生数值，程序依据单位、标定关系和采样时刻解释这个数值。比如记录“25”，尚未说明它是摄氏温度、人数还是某个编号；加入测量对象、单位和时刻后，这组数据才在特定任务中提供清楚的信息。

在这一操作性区分中，信号是承载和传递变化的物理过程，数据是按规则记录的符号或数值，信息是这些记录在给定解释与任务中表达的内容，知识则进一步涉及经组织、理解和验证的关系。四者有联系，边界也随学科而异，不宜写成所有领域通用的唯一哲学定义。一个位串可以被准确传送，却因接收者缺少编码规则而无法理解；理解了句子，也不等于句中断言已被事实证明。

传递过程可以按信源、编码、信道、接收和解释来分析。信源产生待表达的内容，编码把它组织成可传的信号，信道提供传输媒介，接收端按约定恢复符号并解释。储存可看作跨时间保存记录，通信强调跨位置传递记录；两者都依赖物理载体和读写规则。文件被复制后通常仍保留原文件，因此信息复制不能简单套用一个物体从甲处搬到乙处的数量变化。

不同接收端能提取什么，取决于测量灵敏度、编码和处理方法。例如同一份俄语文本，识字者和不懂俄语者对语义的理解不同，但后者仍可能判断字母数量、版面和重复模式。这说明“无法理解语义”不能无条件加强成“完全没有得到任何形式的信息”。讨论原教材的概括性语言时，需要注明正在谈语义、信号还是测量。

1.2 连续、离散与信息处理的层次

物理信号可以随时间连续变化，计算机通常在若干采样时刻记录它，再用有限编码表示每个样本。采样把时间位置离散化，量化把幅值划入有限档位，这两个步骤并非同一件事。一个离散时间序列在数学模型中仍可具有连续的幅值；只有再规定有限数值编码，才成为有限精度的数据表示。

采集过程示例。 假设温度每秒记录一次，并按最近的 $0.1\,^{\circ}\mathrm{C}$ 舍入。采样间隔是一秒，量化间隔是 $0.1\,^{\circ}\mathrm{C}$。理想舍入产生的量化误差绝对值不超过 $0.05\,^{\circ}\mathrm{C}$，但这不包含传感器标定误差和环境变化。提高小数位数不能恢复两次采样之间未测量的过程，加快采样也不能自动提高传感器准确度。文件压缩、传输和显示是后续步骤，还应与最初的信息采集分开。

信息处理在实际系统中相互衔接：采集产生记录，表示规定编码，存储维持可读性，传输连接设备，检索和运算组织数据，解释与验证形成可用结论。温控系统最后还将测量结果与目标比较，生成加热控制量。课本所说的认识功能、交际功能和管理功能，可分别在从数据获得认识、在主体间通信、依据反馈调节行为中找到实例；它们可以同时出现在一个系统中，不是互斥的三种数据格式。

2. 位置记数法与进制转换

十进制中的三百零五表示 $3\times10^{2}+0\times10+5$。以 b 为基数的整数表示也遵循同一结构：

$$(a_k a_{k-1}\cdots a_0)_b=\sum_{j=0}^{k} a_j b^{j},\qquad 0\le a_j<b.$$

二进制只用零和一，十六进制用零至九与 A 至 F。十六进制的一个数字对应四个二进制位，是因为 $16=2^{4}$，并不是因为计算机天生按十六进制计算。十六进制主要让长位串更便于阅读。

位权展开。 $(101101)_2$ 等于 $1\times32+0\times16+1\times8+1\times4+0\times2+1=45$。若把左侧分组补齐为 `0010 1101`，两组分别为二和十三，得到 $(2D)_{16}$。补在最左侧的零不改变非负整数的值，补在最右侧则会乘以二的某次幂，不能混淆。

十进制整数转为基 b 可以反复做带余除法。由 $N=bq+r$ 可知余数 r 是最低位，商 q 包含剩余高位。对商继续同样操作，逐次得到由低到高的数字。

完整转换。 二百五十五除以十六，商十五余十五；十五再除十六，商零余十五。两次余数都是 F，按相反顺序组合为 FF。四十五除二的余数依次为一、零、一、一、零、一，反向读为 `101101`。反向不是任意约定，而是第一次得到最低位这一除法结构的结果。C 语言册中的递归进制输出，正是先处理商再输出当前余数，以恢复高位到低位的顺序。

小数部分则反复乘基数。例如 $0.625\times2=1.25$，取整数位一；$0.25\times2=0.5$，取零；$0.5\times2=1$，取一，故 $(0.625)_{10}=(0.101)_2$。有些十进制有限小数在二进制中无限循环，因此浮点计算通常需要近似表示。

3. 整数编码、溢出与解释类型

八位无符号整数按普通二进制位权表示零至二百五十五，共二百五十六个数。若要在同样的八位中同时表示负数，就必须重新分配编码。补码的一种解释是最高位权为 -2^7，其余位权仍为 2^6 至 2^0，范围为负一百二十八至正一百二十七。

补码分析。 位串 `11111101` 按无符号整数为二百五十三，按八位补码为 $-128+64+32+16+8+4+1=-3$。从正三 `00000011` 出发，逐位取反得 `11111100`，再加一得 `11111101`，也能构造负三。这里改变的是编码解释，不是位串具有两个彼此矛盾的数值。

补码加法可以利用同一套二进制加法电路。`00000101` 与 `11111101` 相加，舍去第九位进位后得到 `00000010`，与五加负三等于二一致。但位宽有限，结果可能超出可表示范围。例如八位补码的一百二十七加一得到位串 `10000000`，按补码解释为负一百二十八，这提示发生了有符号范围溢出。

数学整数没有固定上限，机器整数有。理论中证明 $(a+b)-b=a$，并不能自动保证在某种程序类型中先计算 `a+b` 安全。尤其 C 语言的无符号运算按模数规则处理，而有符号溢出不能直接视作所有实现都保证的回绕。算法、表示、语言语义必须分别检查。

字节序说明多字节对象的各字节按何种顺序存储，不改变一个整数的抽象数值。数字十三的十六进制文本“0D”、内存中的一个字节和字符 `'D'` 的编码是三件不同的事。把打印出来的样子当作内存里逐字符存放的内容，会导致对指针与文件格式的误解。

4. 字符、文本与近似数

字符编码把抽象字符与编号、字节序列联系起来。ASCII 适合其有限字符集合，不能直接覆盖全部汉字和俄文字母。Unicode 规定字符码点体系，UTF-8 则是一种把码点编码为字节序列的方式。码点、字节数、屏幕上的字形数量可能不同，因此“一个字符总是一字节”不能作为一般文本处理规则。

在常见 C 字符串中，末尾的零字节 `\0` 表示结束，它与数字字符 `'0'` 不同；后者在 ASCII 中有自己的非零编码。字符串函数按结束标志处理数据，而不是根据肉眼看到多少个文字。若一个俄语字母在所用编码中占多个字节，按字节截断可能切开编码序列。这是信息表示规则影响程序行为的具体例子。

浮点数以有限位记录符号、指数和有效数字，类似科学记数法。它能覆盖很大范围，却不能精确表示任意实数。十进制零点一在二进制中通常不是有限展开，存入二进制浮点格式时须舍入；若多次运算，误差还可能传播。这里的误差首先来自表示方式，不必归因于程序把加法规则执行错了。

舍入机制的简化演示。 假设一种玩具十进制浮点格式只保留三位有效数字，数一千可写作 1.00×10^3，加一的精确结果为 1.001×10^3，按该精度舍入仍为 1.00×10^3。于是计算结果与一千

相同。这个例子不冒充实际格式的参数，而是说明数量级差异过大时，小增量可能丢失。数值计算册中的误差与条件分析可在这一表示基础上继续展开。

5. 存储程序计算机的组成

处理器执行指令，运行内存保存当前使用的指令、数据及中间状态，长期存储设备保存文件，输入输出设备连接外部信息。存储程序思想把指令也编码存入存储器，处理器按规定的取指、译码和执行过程改变机器状态。程序与数据在物理上都可以是位串，其作用由当前执行上下文决定。

一次赋值的层次。 源程序写 `x = x + 1`，语言层面表示读取变量旧值、加一、再存回。实现层面可能包含从某地址加载、在寄存器中运算、写回内存等操作，具体指令因编译器和体系结构而异。不能凭一条 C 语句断言必然只有一条机器指令，也不能把变量名当作存储器自然存在的标签。

正在运行的程序频繁访问的数据通常位于寄存器、缓存或运行内存等层次；保存后的文件可以位于 SSD、硬盘等长期存储。历史科技俄语句子中的 информация хранится в оперативной памяти 只是在讲运行内存场景，不表示凡出现“存储”就必须选择内存。

机器字长、地址宽度、指令集和操作系统的位数具有联系，却不是可任意互换的标签。课堂板书中的 x86/x64 是体系结构相关记号，不能因其中有字母 x 就识别成异或表达式。研究程序执行时，应先明确正在讨论源语言、机器指令还是硬件电路。

6. 问题规格、算法与程序

给出一个有限数列，求其中最大值。这个问题需要先限定输入非空，否则“最大值”没有定义；还要说明允许哪些数、如何比较、输出什么。问题规格刻画合法输入与正确输出的关系，算法给出求解方法，程序则把算法落实为某种语言和运行环境中的操作。

求最大值的算法可先令当前最大值等于首元素，再逐个读其余元素，遇到更大的便替换。它不是“每个数都取最大”这种循环定义，而是一组可执行的步骤。若输入允许为空，算法必须额外返回失败标志或采用其他明示约定，不能偷偷用零作答案，因为全负数列的最大值并非零。

执行追踪。 对数列 $(-4,-9,-2,-7)$，初始最大值为负四；比较负九后仍为负四；比较负二后更新为负二；比较负七后保持负二，最终输出负二。若初值错误设成零，则所有比较都不会更新，输出不存在于数列中的零。这个反例揭示的不是循环次数问题，而是初始条件不满足算法所需含义。

同一算法可以用 C、俄语伪代码或流程图描述，一个程序也可以同时包含多个算法以及输入输出、错误处理和资源管理代码。算法正确不能替代数组边界、数据类型和接口条件的检查。理论上的任意精度整数运算在 C 的固定宽度类型中实现时，还需要考虑溢出。

7. 循环不变量与终止性

最大值算法为何对所有非空有限数列都正确，不能只靠演示几组输入。处理完前 k 个元素时，当前值应等于这 k 个元素的最大值，这就是循环不变量。它说明循环运行中始终维持的数学关系。

初始化时 $k=1$，当前值为首元素，关系成立。假设处理完前 k 个时成立，下一步比较当前最大值与第 $k+1$ 个：若新数更大则替换，否则不动，两种情况都使新当前值等于前 $k+1$ 个的最大值。归纳后，当全部 n 个元素处理完，当前值就是全体最大值。这三个部分依次为初始化、保持和结束时结论。

部分正确性表示“如果算法终止，输出满足规格”；全正确性还要求在每个合法输入上终止。最大值算法每次少一个未处理元素，未处理数量是非负整数并严格减少，因此有限步后必然结束。这个递减量称为终止度量或变式，不要与每次保持不变的不变量混淆。

欧几里得算法。 对正整数 a,b，反复把 (a,b) 更新为 $(b, a \bmod b)$，直到第二分量为零。其正确性来自 $\gcd(a,b)=\gcd(b,a-qb)$：能同时整除 a,b 的数也能整除余数；反过来能整除 b 与余数的数也能整除 $a=qb+r$。共同因子集合没有改变。

例如 $(48,18)\to(18,12)\to(12,6)\to(6,0)$，输出六。余数始终小于正除数，因此第二分量严格下降直到零，保证终止。这里“最大公约数保持不变”是不变量，“正的第二分量下降”是终止依据，两者分别承担不同的证明任务。

8. 时间与空间复杂度

最大值扫描比较 $n-1$ 次，额外只需一个当前最大值和少数索引，时间随输入长度线性增长，额外空间可为常数。复杂度讨论输入规模增大时资源怎样增长，而不是预测某台机器必定耗时多少秒。

选择排序每轮在剩余部分找极值，比较次数为

$$(n-1)+(n-2)+\cdots+1=\frac{n(n-1)}{2}.$$

因此其通常实现的比较时间为 $O(n^2)$。写两个循环并不自动证明平方复杂度，要看每层循环次数、相互依赖以及循环体成本。若内层每轮只做固定三次操作，即使外观嵌套，总时间也可能线性。

大 O 表示渐近上界，不是精确相等；还应区分最好、最坏和平均情况。二分查找每次将有序候选区间约减半，经过 k 次后剩余约 $n/2^k$，令其降到一可得 k 约为 $\log_2 n$。但它要求能按序比较，并对访问方式有条件；未排序数据不能直接照搬这种查找过程。

可计算性问是否存在满足要求的算法，复杂度问算法需要多少资源。存在指数时间算法的问题仍是可计算问题，实际大规模求解却可能困难。换更快机器会改变常数成本，不能自动消除随规模增长的差异。

9. 状态、字母表与有限自动机

售货设备收到一枚硬币时，是否出货取决于此前积累了多少金额。若只看当前输入而完全不保留过去，就不能正确决定动作。状态把与未来有关的历史压缩成一个有限标记，不需要保存全部过去事件。

形式语言研究符号串。字母表 Σ 是有限符号集合，Σ^* 是全部有限串的集合，包括空串 ε。语言是 Σ^* 的一个子集。自动机读完一个串后接受还是拒绝，就在判断该串是否属于所识别语言。

确定有限自动机定义为

$$M = (Q, \Sigma, \delta, q_0, F), \qquad \delta : Q \times \Sigma \to Q.$$

Q 为有限状态集合，q_0 为初态，$F \subseteq Q$ 为接受状态集合，δ 指定当前状态和当前输入如何决定下一状态。标准 DFA 的转移应对每对状态与符号都有定义；若图上省略某些箭头，须说明它们进入拒绝陷阱状态等约定。

控制设备的状态机还可能输出“开门、锁门、显示错误”等动作；用于语言识别的五元组只规定接受性，不能把它与所有工业控制模型完全等同。二者共享状态转换思想，但建模目标不同。

10. 有限自动机的构造与正确性

考虑识别二进制串中一的个数为偶数的语言。只需要区分“至今偶数个一”和“至今奇数个一”，设状态为 E、O。空串含零个一，因此初态 E，也是接受状态；读零不改变奇偶性，读一交换 E 与 O。

转移规则为 $\delta(E,0)=E$、$\delta(E,1)=O$、$\delta(O,0)=O$、$\delta(O,1)=E$。这些规则覆盖两个状态与两个输入的全部组合。

完整运行。 输入 `1011`，从 E 读第一个一到 O；读零仍在 O；读第二个一回 E；读第三个一到 O。读完停在非接受态，故拒绝。输入 `1100` 则经过 E、O、E、E、E，最终接受。空串不读任何输入，留在 E，也被接受。

证明与循环不变量完全相通：处理任何前缀后，状态 E 当且仅当该前缀含偶数个一。空前缀满足；追加零保持数量，追加一改变奇偶，转移恰好维持这一意义。因此读完整串时接受与题意严格一致。只画一张状态图而不解释状态含义，容易得到看似合理却无法证明的设计。

再考虑识别以 `01` 结尾的串。可用三个状态：S 表示尚无有用后缀，A 表示末尾是零，B 表示末尾为零一并接受。S 读零到 A、读一留 S；A 读零留 A、读一到 B；B 读零到 A、读一到 S。

状态不是简单记录“已经看到过零一”，因为题目要求结尾而非中间出现。`011` 先到 B，又读一回 S，最终拒绝，正好体现这一区别。

10.1 非确定转移与模拟

非确定有限自动机允许在同一状态读同一符号时有多个后继，也可以按所用定义允许不消耗输入的空转移。它接受一个串，意味着至少存在一条完整运行路径在读完该串后到达接受态，并不要求所有路径都接受。这里的“非确定”是数学定义，不是设备随机出故障，也不必用概率来解释。

可以用一个状态集合记录当前所有可能状态：每读一个符号，分别求各可能状态的后继，再合并成新的集合；含有接受状态的集合就是接受集合状态。若原自动机有有限个状态，其状态子集也只有有限多个，因此可以构造等价 DFA。非确定有限自动机可能让描述更简洁，却没有突破有限自动机能够识别的语言范围。

这也说明“分支更多”不自动等于“能解决更强的问题”。比较计算模型能力，需要证明它们能否互相模拟，或找出一方可识别、另一方不可识别的语言，不能只凭图面复杂程度判断。

10.2 子集构造、空转移与状态数上界

设 NFA 有 n 个状态，模拟状态是其可能状态的子集，因此最多有 2^n 个候选子集；这只是上界，许多子集不可达，不必全部建立。空集合也有明确含义：当前没有任何可能运行。此后再读符号仍为空，可作为 DFA 的拒绝陷阱状态。

若允许 ε 转移，先定义 $\mathrm{cl}_\varepsilon(S)$ 为从 S 中各状态出发、经过零次或多次空转移可达的全部状态。初始子集为 $\mathrm{cl}_\varepsilon(\{q_0\})$。读符号 a 后更新为

$$S' = \mathrm{cl}_\varepsilon\left(\bigcup_{q \in S} \delta(q, a)\right).$$

由于进入这一步之前 S 已经闭包化，先把所有读 a 的后继并起来，再补空转移闭包即可。新子集与原接受集合相交非空时接受；“全部状态都是接受态”是另一种更强的要求，不符合 NFA 的存在一条接受路径语义。

正确性可沿输入长度归纳：初始子集正是读空串可达的状态；每次更新保留且仅保留读完新前缀的所有可能状态。因此最终子集含接受态，恰等价于原 NFA 有一条读完整串的接受路径。状态数上界与语言能力分别回答描述规模和可识别范围，不应混成一个指标。

10.3 补、交与并的自动机构造

对转移完整的 DFA，交换接受态与非接受态，得到相对同一字母表的补语言。每个输入串原来恰有一条最终运行，交换接受性便逐串取反。若图中存在省略的转移，应先补拒绝陷阱；直接交换已有圈的标记可能遗漏那些本来无路径的串。对 NFA 也不能直接交换终态，因为“至少一条接受路径”的否定是“每条路径都不接受”，不是“至少一条路径到达原非终态”。

两台同字母表的 DFA 可组成乘积。状态为 (q_1,q_2)，同步读取一个符号 a：

$$\delta((q_1,q_2),a)=(\delta_1(q_1,a),\delta_2(q_2,a)).$$

若要交语言，接受条件为 $q_1\in F_1$ 且 $q_2\in F_2$；若要并语言，则改为至少一个属于接受集合。状态上界为 $|Q_1||Q_2|$，同样可删除不可达状态。

讲解例：同时要求偶数个一且以01结尾。 用本章的奇偶机与后缀机组成乘积，初态为 (E,S)，交语言只有 (E,B) 接受。输入1001依次到 $(O,S),(O,A),(O,A),(E,B)$，最终接受。输入1101虽然以01结尾，一的数量为三，最终为 (O,B)，拒绝。状态的两个分量分别维持各自不变量，因此不需要重新猜一套没有语义的六个状态。

11. 有限记忆的边界

DFA 可以读取任意长的串，但内部状态数固定。长度没有上限，不表示可以记住任意大的精确计数。奇偶只需两个状态，因为未来判断只需要余二的信息；要求精确匹配两个任意大数量则不同。

语言 $L=\{a^nb^n\mid n\ge 0\}$ 要求先出现若干 a，随后同样数量 b。假设存在具有 k 个状态的 DFA 识别它，考察读完 $\varepsilon,a,a^2,\dots,a^k$ 这 $k+1$ 个前缀后的状态。根据抽屉原理，必有 $i<j$ 使读完 a^i 与 a^j 后状态相同。

此后给两者都接上 b^i。由于起点状态相同、后续输入相同，确定自动机最终接受性相同。然而 a^ib^i 属于语言，a^jb^i 不属于语言，产生矛盾。故不存在这样的 DFA。这个证明具体指出有限状态何处丢失了必要信息，而不是仅说“机器记忆不够”。

若预先规定 $n\le 100$，语言变成有限集合，可以构造足够大的 DFA 处理。理论结论依赖输入不设固定上界，不能把它误读成任何现实中有限长度的匹配任务都无法完成。下推自动机用栈保存更多信息，能处理这一典型匹配语言；图灵机则进一步允许一般读写存储。

12. 图灵机的计算机制

图灵机包含有限控制状态、带字母表、可读写带、读写头和转移规则。标准单带模型中，规则根据当前状态与读到的符号，指定写入符号、向左或右移动以及下一状态；停机接受或拒绝状态不再继续普通转移。带在理论上没有预设固定长度，但一次有限计算只访问有限区域。

可以用配置记录整个计算瞬间：当前状态、读写头位置和带上非空内容。一步转移把一个配置变成下一个配置。程序执行中的变量和程序位置也是机器状态的一部分，因此“有限控制器加可扩展存储”比只有有限个状态的自动机具有更强表达能力。

一元数加一。 用连续 n 个符号一表示自然数 n，读写头起初位于最左符号；零用空白带表示。取扫描状态 q：读到一，原样写一、向右、保持 q；读到空白，写一并进入停机状态。输入

111 时连续跨过三个一，遇到右侧空白写一，得到 1111。输入零时第一格即空白，写一得到 1。算法覆盖零与正数，不需要把空输入留成未定义情况。

终止依据是输入最初只有有限多个一，读写头每步向右，最终必遇到第一个空白；正确性依据是原有一从未删除或改变，只在末尾增加一个。若输入允许出现其他符号，需要事先规定拒绝规则，不能把它们也默认为合法一元数。

匹配语言的思路。 对 $a^n b^n$，图灵机可先检查串是否由一段 a 接一段 b 组成；再把最左未标记 a 改为 X，向右找到一个未标记 b 改为 Y，再回到左侧重复。若某次找不到所需 b，拒绝；若所有 a 用完后仍有未标记 b，也拒绝；恰好配完则接受。每轮减少一个未标记 a，扫描又限定在有限输入区间及邻近空白，故整个过程有限。新增的标记和反复往返提供了 DFA 没有的可扩展记忆。

13. 通用计算与停机问题

如果把一台图灵机的状态、符号和转移表编码为有限串，另一台机器可以读取这段编码并模拟它。这就是通用机器的基本想法。现代计算机存储程序并执行不同程序，与这种可编码、可模拟的思想有联系，但通用性不表示所有数学问题都有算法。

停机问题询问：给定程序 P 和输入 x，$P(x)$ 最终是否停止？它要求一个统一判定器在每一种合法编码输入上都给出正确答案，并且自身总能停止。逐步模拟能发现已经停机的情况，却不能仅凭“至今没有停止”证明以后永不停止。

对角论证。 假设存在总正确且总停止的判定器 $H(P, x)$。构造程序 D，以程序编码 y 为输入：若 H 判断 y 运行自身会停机，D 就进入无限循环；若 H 判断不会停机，D 就立刻停止。D 本身也是程序，具有可输入的编码，因此可以研究 D(D)。

若 H 判断 D(D) 会停机，按 D 的定义它会无限循环，H 错；若 H 判断不会停机，D 又立刻停止，H 仍错。两种可能都矛盾，因此假设的统一判定器不存在。证明依赖程序可以被编码并作为输入，不是一个脱离机器规则的语言悖论。

这个结论不意味着任何具体程序的终止都无法分析。最大值扫描和欧几里得算法已经给出了终止证明；固定次数循环也容易分析。不可判定否定的是覆盖全部程序与全部输入的万能总算法，不否定受限类别、具体实例和部分自动分析工具。

14. 抽象层次之间的联系

信息编码规定同一位串如何解释；逻辑函数规定输入位怎样映射到输出位；存储与状态让结果依赖历史；算法说明求解步骤及其正确性；程序选择类型和运行接口；计算模型研究哪些问题原则上可解，复杂度再研究资源需求。这些层次构成相互支持的知识结构，不能用某一条课堂箭头替代全部定义。

例如一个自动售货程序可以先用 DFA 描述控制状态，再用寄存器保存状态编码，用逻辑电路或程序计算下一状态。若系统需要记录无固定上限的账户数据，有限控制图只能描述部分流程，还要把数据存储纳入模型。模型选择取决于问题需要保留什么信息，而不是看图是否画得简单。

本册依据可读的计算基础白板讲解、进制转换、科技俄语设备概念和 C 语言算法对话编排。历史标题为图片生成的会话，正文确有计算基础内容；但白板原图在导出文本中未完整保留，本册不冒称重新读到了每一处字迹。状态机、图灵机操作及停机论证的详细例子为教学重构，信息表示是其先修基础，具体 C 实现与逻辑门实现分别衔接本套对应分册。